

A population-size fidelity framework for reference-vector multi-objective evolutionary algorithms with an audit of PlatEMO

Yuxuan ZHANG^{1*}

¹Graduate School, Army Engineering University of PLA, Nanjing 210007, China

Abstract Reference-vector multi-objective evolutionary algorithms derive their population from a reference-vector constructor, and a comparison is only credible if the size used is the size reported. We show that in PlatEMO 4.16 this can fail silently, and turn it into a reusable audit. The framework separates three quantities, requested, evaluated and recorded, and providing a fidelity principle with two propositions, an audit, a repair catalogue and a reporting protocol. The lexical audit finds the assignment in 84 of 1887 source files, belonging to 83 of the platform's 360 algorithms. A trace of sixteen algorithms shows the consequence: configuring 100 realises 91 individuals for nine of the thirteen affected algorithms, as few as 31 for the four that shrink their reference set, and 100 for three unaffected ones, so a table reporting 100 throughout compares populations differing by up to a factor of three. The platform records the constructor's cardinality rather than the request, without flagging the divergence. On a fifteen-problem suite, restoring the requested size improves the median indicator on twelve problems, barely on a thirteenth and not on two, and explains most of the median scheduling gain; three repair cases occur in practice, so no repair is universal.

Keywords population-size fidelity, reference-vector algorithm, benchmarking reproducibility, evolutionary multi-objective optimization, PlatEMO, fair comparison, large-scale optimization

Citation A population-size fidelity framework for reference-vector MOEAs. Sci China Inf Sci, for review

1 Introduction

Decomposition-based and reference-vector multi-objective evolutionary algorithms (MOEAs) dominate current practice, and their population is organised around a set of reference vectors whose cardinality is taken to be the population size N [1–3]. The field is moving quickly on two fronts at once. Large language models are being woven into the evolutionary loop as offspring generators, operator designers and hyper-heuristics, and the first surveys of this intersection have appeared [4–6]. In parallel, large-scale and many-objective problems keep raising what a reference-vector method must deliver, as the most recent surveys of large-scale multi-objective optimization make clear [7, 8]. Across both fronts one quantity is quietly load-bearing: the population size. It fixes the number of reference vectors and therefore the resolution of the search, it fixes the evaluation budget consumed per generation and therefore how many generations fit into a fixed budget, and it is reported in every experimental section as a controlled quantity. Whether a comparison is fair hinges on it, a point made explicitly in recent community guidance on fair benchmarking of evolutionary multi-objective algorithms, which names the population size among the few experimental choices that most distort a comparison [9, 10].

We report that in a widely used platform this controlled quantity is not, in general, what it appears to be, and we argue that the underlying failure mode is general enough to deserve a framework. In PlatEMO [11], a reference-vector set is obtained by calling a constructor that returns both the vectors and their number. A large number of algorithm files pass the user's population size to that constructor and then assign the second output back to the population size, so that the requested size is silently replaced by the size of the reference lattice. The constructor does not return the requested size: over the range $2 \leq M \leq 15$, $M \leq N \leq 1000$, it never returns more points than requested and returns exactly the requested number in only 8.7% of cases. At $N = 100$ the platform therefore runs with 91 individuals at three objectives and 65 at ten objectives, a reduction of 9% and 35% with no message to the user. The same mechanism can be present in any library that returns a reference set together with a size that is then written into a shared problem object.

* Corresponding author (email: 3150644070@qq.com)

Three quantities have to be kept apart, because the whole failure lives in the gap between them. The first is the size a study requests and declares, N_{req} . The second is the size the method actually evaluates, N_{real} . The third is the size the run records, N_{rep} . We traced all three in the platform studied here (Table 2), and the outcome is unambiguous: the constructor returns 91 for a request of 100, and nine of the thirteen affected algorithms then evaluate exactly those 91 individuals, so for those nine $N_{\text{real}} = N_{\text{rep}} = 91$ while $N_{\text{req}} = 100$; the other four evaluate fewer, as Table 2 shows. For those nine, the platform records the constructor’s cardinality faithfully: it is not inventing a number, it is recording what the constructor returned. What it never records is the size the study asked for, because the assignment has overwritten it. For the other four algorithms, which shrink their own reference set during the search, it does not record the size actually used either, so there the recorded figure overstates the population. What is missing throughout is any signal that the request was not honoured, and the consequence is a confounded comparison rather than a corrupted result. In the same release, NSGA-II, SPEA2 and IBEA contain no such assignment and run at exactly 100, so an experimental table that reports $N = 100$ for NSGA-III and for NSGA-II is comparing 91 individuals with 100, and a table that mixes MOEA/D with SPEA2, or RVEA with IBEA, is comparing 91, 83 or 31 with 100. The nominal size in the table is the same; the runs are not.

That is the failure the framework addresses, and it is a reporting failure rather than a coding mistake. The rounding itself is unavoidable: a symmetric lattice in M objectives admits only certain cardinalities, so a request that falls between two admissible values has to be rounded. What is avoidable is that the rounded value replaces the requested one without the study being told, so that the number printed in the experimental section is no longer the number the run used. It follows that the remedy is not to forbid the write-back, which many algorithms use deliberately to keep the population and the reference set consistent, but to make the divergence visible and to report the size that was realised.

The size actually realised leaves traces that a reader can check without instrumenting the platform, which is what makes the problem detectable rather than merely suspected. The per-generation cost of a reference-vector method is proportional to N_{real} , so under a fixed evaluation budget the number of completed generations is inversely proportional to it; in our controlled runs the declared size of 100 corresponds to 499 generations and the realised size of 91 to 552, a ratio of 1.106 against the predicted 1.099. A study that reports its generation count, or publishes convergence curves against generations, therefore exposes the discrepancy, and a study that reports only the final indicator hides it.

The mechanism is easy to miss, and we missed it ourselves: it surfaced only when a controlled comparison of an adaptive scheduling layer refused to reproduce. An earlier version of that layer appeared to improve its host algorithm by a double-digit percentage, and the improvement survived visual inspection of convergence curves. It did not survive matching the population size, because the host was running with 91 individuals while the layer ran with 100.

We therefore do not present the episode as a bug report. We abstract it into a framework for *population-size fidelity*: the property that the population size a method runs with equals the size it reports. The framework consists of four parts. First, a fidelity principle that states what must hold and why a violation is a threat to reproducible comparison, since reproducibility in evolutionary computation is itself an established concern [12]. Second, an auditing procedure that locates the offending constructor calls in a code base. Third, a catalogue of corrections, which shows that a single one-line repair is not universally valid and that the admissible repair depends on how an algorithm couples its population to its reference set. Fourth, a reporting protocol that makes violations visible in future work. We then apply the whole framework to PlatEMO 4.16 as a case study, quantify the consequences with controlled experiments on a large-scale suite, and show that the framework is what turns an isolated observation into a checkable, reusable procedure.

The contribution is therefore not the disclosure of a defect but the construction and validation of an instrument. Specifically, the paper (i) separates the requested, realised and reported sizes and formalises fidelity over them, proving that the deficit introduced by a lattice constructor is one-signed and inflates the number of generations completed under a fixed budget; (ii) reduces the search for infidelity to an auditable pattern, tested against fourteen positive and negative cases; (iii) shows by construction that the repair is algorithm-dependent, with three cases realised by real algorithms and the mechanism of the hardest case measured rather than asserted; (iv) quantifies the consequences on a fifteen-problem suite with a paired design, effect sizes and a three-arm control that separates the two effects a single comparison conflates; and (v) supplies a reporting protocol together with fully released materials.

2 Related work

Four strands of recent work frame this study. The first is the reference-vector family itself. Reference-vector guided selection remains the workhorse for many-objective problems, and recent methods continue to build on it, for example by adapting the reference vectors to sparse or irregular fronts and by replacing the convergence measure used inside selection [13]. In all of these methods the population size is the hidden constant, because the reference set and the population are indexed by the same N .

The second strand is the benchmarking-methodology literature. Community guidance has long warned that an indicator value is comparable only when the experimental protocol is fixed [14, 15]. Comparison results have since been shown to depend strongly on the specification of the population size, the quantity we audit [9]; wall-clock time rather than evaluations alone has been argued to be the budget that matters in practice [16]; the reproducibility of evolutionary computation has been examined in its own right [12]; the rigour of the field has been critically reviewed [17]; and the design of benchmarks with practical impact has been reconsidered [18]. The closest antecedent to our work is an analysis of single-objective algorithms implemented across thirteen frameworks, which found that even identical parameter settings can yield different results because of implementation differences; that study raises the general concern and calls for guidelines, but it does not isolate a mechanism that a reader can check, and it covers single-objective algorithms [10]. Our contribution is a mechanism of exactly that kind, located in the reference-vector constructor, together with the audit and the repairs that make it actionable.

The third strand is the rapid arrival of methods driven by large language models. Recent work uses large language models to generate offspring, to design operators and to steer multi-objective search end to end [4–6, 19], and these methods are almost always benchmarked against reference-vector baselines drawn from the same platforms that we audit. A systematic population-size mismatch in those baselines would therefore propagate into the comparisons that this fast-moving literature uses to establish its gains.

The fourth strand is the widening of the problem classes to which reference-vector methods are applied. Surrogate-assisted methods for expensive and high-dimensional problems, and constrained variants, are surveyed at increasing frequency [20–22]; every one of these families inherits the reference-vector selection machinery and, with it, the population-size convention under audit. The fidelity problem is therefore not tied to one benchmark but to the shared substrate of a whole class of methods.

3 The population-size fidelity framework

We now state the framework in a form that is independent of any particular library. Throughout, a *reference-vector method* is an algorithm whose population is organised around a finite set of reference vectors, the symbol N denotes the population size that the user sets, and M denotes the number of objectives.

3.1 The fidelity principle

We first make the notion precise, because the whole framework rests on distinguishing three quantities that are usually conflated.

Definition 1 (Population-size fidelity). Consider a reference-vector method configured with a requested size N_{req} , evaluating a population P of size $N_{\text{real}} = |P|$, and recording a size N_{rep} in its own output. The method is *population-size faithful* if $N_{\text{req}} = N_{\text{real}} = N_{\text{rep}}$. Otherwise it suffers a *size infidelity*, and we call $\delta = N_{\text{req}} - N_{\text{real}}$ its *size drift*.

The definition isolates the three quantities deliberately, because a run can satisfy any two of them while violating the third. Three variants of the failure are worth separating. A run that rounds its request and then records the rounded value is honest about what it did: it is unfaithful only in the sense that the request could not be honoured, and the discrepancy is found by comparing N_{req} with N_{real} before the run rather than after it. A run that rounds, records the rounded value faithfully, and is then written up in a study whose tables state the original request is the case that threatens a comparison; there the platform is right and the paper is not, and the discrepancy is found by comparing the number in the table with the number the run realised. A run whose population changes again after the constructor has returned — because the algorithm prunes its own reference set — leaves a record that matches neither the request nor the population, and then the platform itself overstates the run. Section 4 shows that PlatEMO produces the second variant for all affected algorithms and the third variant as well for four of them. All three quantities are observable without modifying the method or the library: N_{req} is what the study configures, N_{real} follows from the cardinality of the reference set and the evaluation cost of a generation, and N_{rep} is what the run stores. This is why an audit can rest on the definition rather than on the intent of the code.

Fidelity matters for three reasons. First, the realised population fixes the cardinality of the reference set and hence the resolution of the search. Second, it fixes the evaluation budget consumed per generation and therefore the number of generations that fit into a fixed budget. Third, the reported size is the quantity a reader uses to judge whether two methods were compared under the same conditions, and this third reason is the most insidious, because an unfaithful run still produces a valid Pareto approximation and raises no error.

It is useful to separate fidelity from quality. An unfaithful run is not necessarily a poor run, and a faithful run is not necessarily a good one. Fidelity is the weaker property that the experiment is what it claims to be, and it is exactly the property that a comparison across methods presupposes. It follows that restoring fidelity does not in itself improve an algorithm: it changes what the reported number means. The empirical sections below are written with this distinction in view, and we therefore report the direction and the size of every difference rather than treating fidelity as a performance claim.

The practical interest of the definition comes from the following two facts, which are what convert an anecdote into a checkable statement.

Proposition 1 (Monotone downward drift). Let $L(N, M)$ denote the number of rows of the matrix returned by the two-layer lattice constructor used in this study (Section 4) for a requested size N and M objectives; this is the cardinality the calling algorithm sees, and the size it writes back into the population field. Then $L(N, M) \leq N$ for all $M \geq 2$, $N \geq M$, with equality if and only if N is an admissible lattice size; consequently $\delta \geq 0$: the drift can only remove individuals, never add them.

Proof. The constructor builds a first layer from the integer H_1 , the smallest value with $\binom{H_1+M}{M-1} > N$. Because $H_1 - 1$ fails that inequality, the layer contains exactly $\binom{H_1+M-1}{M-1} \leq N$ rows; when $H_1 = 1$ the layer contains $\binom{M}{M-1} = M \leq N$ rows, the boundary case being $M = N$. A second layer is added only if $H_1 < M$ and the first layer is short of N ; its size $H_2 \geq 1$ is the smallest integer with $\binom{H_1+M-1}{M-1} + \binom{H_2+M}{M-1} > N$. Hence $H_2 - 1$ satisfies the opposite inequality, and the second layer actually contributes $\binom{H_2+M-1}{M-1}$ rows (where $H_2 + M - 1 = (H_2 - 1) + M$, since H_2 is the smallest integer that exceeds the budget and $H_2 - 1$ is the largest that does not), exactly the binomial coefficient that the stopping condition bounds by $N - \binom{H_1+M-1}{M-1}$. The total is therefore at most N and never exceeds the request. Equality requires that the stopping condition be met at the step that exhausts the request, that is, that N coincide with an admissible lattice size; otherwise the constructor returns the largest admissible size below N . A duplicated row between the two layers would reduce the number of distinct directions but not the row count that $L(N, M)$ measures, so it cannot affect the inequality; we verified that no duplicate occurs over the tested range.

We verified the proposition exhaustively rather than trusting the argument alone. Over $2 \leq M \leq 15$ and $M \leq N \leq 1000$, that is 13895 distinct combinations, the realised count exceeds the request in 0 cases, and the two layers never yield a duplicated direction. The counts were also checked against the platform's own constructor at 39 points spanning the range and at all 81 cells of the grid reported in Table 3, with no disagreement.

The range is the range we scanned, not a documented limit: the platform's problems take the number of objectives as an argument, and its sources carry defaults up to ten (for example, the MaF8 problem declares `obj.M = 10` as its default). It brackets the region in which reference-vector methods are actually run and is cheap enough to enumerate completely. The complete table $L(N, M)$ over that range is released as a machine-readable file, and points outside it follow from the closed form of the constructor rather than from a measurement.

Proposition 2 (Generation inflation). Consider a method whose report and printed tables state a size $N_{\text{req}} = N$, but which evaluates a population of size $N_{\text{real}} = L < N$ and consumes cL evaluations per generation for a constant $c > 0$ fixed by the method, stopping when the evaluation budget FE_{max} is exhausted. Then it completes $\lfloor \text{FE}_{\text{max}}/(cL) \rfloor$ generations where the reported size implies $\lfloor \text{FE}_{\text{max}}/(cN) \rfloor$, a relative excess of $(N - L)/L$ in the number of generations.

Proof. The number of generations is the budget divided by the cost of a generation, which is cN_{real} . Substituting $N_{\text{real}} = L$ for the actual run and $N_{\text{req}} = N$ for the reported condition, and taking the ratio, gives N/L , that is, an excess of $(N - L)/L$.

The premise of Proposition 2 is the reporting condition and not the internal mechanics: the excess appears whenever the quantity a study states is N_{req} while the quantity the run realises is L . A run that both realises L and reports L is described by the same formula with $N = L$ and shows no excess, which is the consistency condition of the framework. Two consequences follow. When the constructor is the one of Section 4, Proposition 1 makes the drift one-signed, so an unfaithful method is biased in a fixed direction rather than randomly perturbed. When the comparison is made under a fixed evaluation budget, Proposition 2 makes the drift translate directly into a difference in the number of generations, which is exactly the quantity that a convergence curve displays. Neither fact is visible from a single reported population size, which is why the framework is needed.

3.2 Auditing a library for size infidelity

The audit rests on a single observation: size infidelity arises when the output of a reference-vector constructor is written into a shared object that stores the population size. Algorithm 1 formalises the procedure.

Algorithm 1 Auditing a library for population-size infidelity

Require: source tree $\mathcal{T}$; set of constructors $\mathcal{C}$ with documented behaviour
Ensure: census of unfaithful algorithms and files; drift function $\delta(N, M)$

- 1: $S \leftarrow$ calls to any $\phi \in \mathcal{C}$ whose return list contains the population-size field
- 2: **for** each call $s \in S$ **do**
- 3: **if** ϕ returns exactly the requested size **then**
- 4: classify s as faithful; continue
- 5: **end if**
- 6: evaluate $\delta_s(N, M) = N - L_\phi(N, M)$ over the documented range of N and M
- 7: record the file of s and the sign of δ_s
- 8: **end for**
- 9: return the census of calls with $\delta_s > 0$, grouped by algorithm family, together with δ

Step 3 is the only step that requires judgement, and it depends only on the documented behaviour of the constructor rather than on the code that calls it, so the census is reproducible by a reader who has the same release. In the case study of Section 4 the constructor is a lattice whose cardinality is the largest admissible value not exceeding the request, which makes δ a deterministic function that can be tabulated once and reused; by Proposition 1 the sign of δ is non-negative throughout.

Two implementation choices decide whether the census is trustworthy, and we make both explicit because a naive search over source text is not. First, the match must not be satisfied by a comment or by a string literal, since algorithm files document the pattern in comments and build expressions with `eval` and `sprintf`. We therefore run the pattern over a lexically reduced form of each file in which line comments, block comments and single- and double-quoted literals have been replaced by blanks, with line and column positions preserved, and we additionally require the hit to sit on a live statement. This is a lexical and not a syntactic analysis, so it can in principle miss a call assembled dynamically. We therefore wrote a suite of fourteen positive and negative constructions, covering the constructions a MATLAB source file can use to hide such a call: comments, block comments, both quote styles, escaped quotes, a percent sign inside a string, a transpose before the match, a line continuation, an assignment to another field, and calls assembled by `eval` and by `sprintf`. Twelve behave as intended and the two dynamic constructions cannot be seen lexically at all; they are disclosed as limitations. The suite earned its keep: it showed that the parser initially missed a call split across a line continuation, which we fixed, and the census is unchanged by that fix, so the gap was a false-negative path rather than a miscount. Because a miss under-counts, we treat the census as a lower bound, and we inspected the 84 surviving hits by hand. The parser and the suite ship with the release.

Second, the audit records three quantities and not one: the requested size, the cardinality the constructor returns, and the population the run actually evaluates. Recording only the first would miss the drift, and recording only the first two would miss the four algorithms of Section 4 whose population changes again after the constructor has returned.

3.3 A catalogue of corrections

Restoring fidelity is not a single edit, because algorithms couple their population to their reference set in different ways. We distinguish three cases, and for each we state the edit, the constructor requirement and the test that fails before the edit and passes after it.

Case A: directions only. The reference vectors are used only to define directions, and the number of individuals need not equal the number of vectors. The edit discards the second return value of the constructor, `[Z, ~] = UniformPoint(N, M);`, so that the reference set is left untouched while the population keeps the requested size. The applicable test is that `size(Population, 1)` equals the requested N at the end of the first generation while the reference set has $L < N$ rows.

Case B: explicit indexing. The algorithm indexes its individuals by reference vector, so the population must equal the reference set. Discarding the size output breaks the indexing, and the run fails with an index-out-of-bounds error, which is the test for the case; the constructor must instead be replaced by one that returns exactly N points. A minimal exactly- N constructor is a lattice of the largest admissible size below N , padded with the $N - L$ points that maximise the minimum distance to the points already chosen, with a de-duplication step and an assertion that

Table 1 Smallest pairwise angle between the directions of the reference set at a requested size N . The default constructor returns $L < N$ directions; the exactly- N constructor returns N but reduces the angular resolution

N	M	default $\gamma_{\min}$	exactly- N $\gamma_{\min}$	ratio
100	3	5.19°	3.57°	1.45
100	5	18.19°	7.89°	2.31
100	8	16.38°	13.11°	1.25
300	3	2.60°	1.65°	1.58

the number of rows equals N ; because the padding is added after the lattice, the reference set changes as well as the population, which is why this repair is not interchangeable with case A.

Case C: normalisation by inter-vector geometry. The algorithm normalises a selection criterion by the smallest angle between neighbouring reference vectors. Here neither previous repair is admissible without additional care, and the reason can be measured. Table 1 reports the smallest pairwise angle of the two constructors. The exactly- N constructor does not collapse the angle to zero, but it reduces it, by a factor of 1.45 at $N = 100$, $M = 3$ (5.19° to 3.57°) and by a factor of 2.31 at $M = 5$ (18.19° to 7.89°). Since the criterion is divided by this angle, the change is a proportional amplification of the convergence term rather than a singularity, and a repair for this case has to bound the angle explicitly rather than merely return N points. A minimal admissible construction is therefore: take the largest admissible lattice $L \leq N$, then add the $N - L$ remaining directions one at a time, each chosen to maximise the smallest angle to the directions already present, and reject a candidate whose inclusion would drop that angle below a floor $\theta_{\min} = 1^\circ$; if the floor cannot be met, fall back to $N = L$ and report the fallback rather than silently shrinking the population.

We implemented this construction and measured it. At $N = 100$ with three, five and eight objectives it returns exactly 100 directions while retaining the smallest angle of the underlying lattice (5.19°, 18.19°, 16.38°), where the plain exactly- N constructor of Table 1 degrades them to 3.57°, 7.89° and 13.11°; the floor was never binding, so the fallback never fired (Table S4 records the four settings tested). The applicable test is therefore two-sided and both halves are met: the minimum pairwise angle of the repaired set is at least $\theta_{\min}$, and the cardinality equals the requested N .

The catalogue is the reason a framework is needed at all: a reader who adopts “the one-line repair” as a universal rule will silently break the algorithms in cases B and C, and one who adopts the exactly- N constructor as a universal rule will reduce the angular resolution of those in case C by up to a factor of 2.31. Section 5.6 exhibits all three cases on real algorithms.

3.4 A reporting protocol

Finally, the framework prescribes what a study should report, and it prescribes it as a schema rather than as advice, because advice is not adoptable. A reference-vector study should print, for every configuration it compares, the seven columns

`N_req | N_real | N_rep | constructor | repair | generations | FE_used ,`

with three decision rules attached, and with the check expressed as code so that it can be run rather than remembered. Rule 1: if $N_{\text{req}} \neq N_{\text{real}}$, the row must say so, and any claim about the method must use N_{real} . Rule 2: if $N_{\text{rep}} \neq N_{\text{real}}$, the platform’s record is not the population that was used, so the row must report all three quantities and no downstream claim may rely on the record; Table 2 shows that four of the sixteen traced algorithms are in this state. Rule 3: a comparison table must not mix rows whose `N_real` differ unless the difference is the object of study, which Table 2 shows is violated by default. The protocol costs one line in the experimental section and one assertion at the end of a run, `assert(numel(Population) == N_real && Problem.N == N_real)`. The protocol deliberately does not require suppressing the write-back of the size into the shared problem object, which many algorithms use to keep the population and the reference set consistent. What it requires is that the number in the table be the number the run used, and that an unrealisable request be declared as such before the runs rather than discovered afterwards. A study that cannot realise a request should either move the request to the nearest admissible value, as we do for the declared arm of Section 5, or state the realised value and treat the request as nominal.

Table 2 Trace of the three quantities for 16 reference-vector algorithms of PlatEMO 4.16, all configured with $N_{\text{req}} = 100$ at $M = 3$. L is the cardinality returned by the constructor, N_{real} the population present in the run’s own result, and N_{rep} the value held in the platform’s population-size field after the run. N_{rep} equals L for every algorithm, so the recorded value is the constructor’s cardinality rather than the realised population wherever the two differ. The last three algorithms carry no such assignment and serve as controls

algorithm	N_{req}	L	N_{real}	N_{rep}	status
NSGA-III	100	91	91	91	diverges
MOEA/D	100	91	91	91	diverges
RVEA	100	91	83	91	diverges
LMOCSSO	100	91	66	91	diverges
FDSEA	100	91	91	91	diverges
HEA	100	91	91	91	diverges
C-TAEA	100	91	91	91	diverges
MOEA/D-DRA	100	91	91	91	diverges
MOEA/D-DE	100	91	91	91	diverges
RVEAa	100	91	31	91	diverges
DGEA	100	91	80	91	diverges
PPS	100	91	91	91	diverges
MOEA/D-STM	100	91	91	91	diverges
NSGA-II	100	–	100	100	faithful
SPEA2	100	–	100	100	faithful
IBEA	100	–	100	100	faithful

4 Case study: auditing PlatEMO

4.1 The override

We audit PlatEMO 4.16, released in July 2026 with 360 algorithms and 630 problems; this is the version in which the host algorithm used in Section 5 was added, and its release note is the point of reference for the census below. The affected line is

```
[Z,Problem.N] = UniformPoint(Problem.N,Problem.M);
```

in which the second output of the constructor [23] is written into the population size. The constructor in question is the classic two-layer simplex lattice of Das and Dennis [24], and the rounding it performs is a mathematical property of that lattice rather than an implementation defect: a symmetric lattice in M objectives has a cardinality that grows as a binomial coefficient, so only certain counts are realisable and any other request has to be rounded down to the nearest admissible value. We take this as given and do not treat it as the subject of the paper. What we examine is the write-back: the rounded value is assigned to the field that stores the user’s request. Table 2 traces the three quantities for 16 reference-vector algorithms of the release, all launched with a requested size of 100 at three objectives. Two facts follow. First, the write-back is real and it propagates: because **PROBLEM** is a handle class, the assignment mutates the caller’s object, so after the run the platform’s own population-size field holds 91 rather than the requested 100. The platform therefore records the cardinality the constructor returned, and it records it faithfully; what it never records is the request, which the assignment has overwritten. Second, and more consequentially, that recorded value is not always the population that is used. For nine of the thirteen affected algorithms the two coincide at 91, but RVEA finishes its run with 83 individuals, DGEA with 80, LMOCSSO with 66 and RVEAa with 31, because those methods prune or re-select their reference set during the search, and the recorded field still reads 91. For those four algorithms the platform’s own record therefore overstates the population, by up to a factor of three. NSGA-II, SPEA2 and IBEA are unaffected and run at the requested 100. A study that compares any two of these at a nominal $N = 100$ is therefore comparing populations that differ by up to a factor of three. Two properties of the default constructor decide the direction of the divergence. First, by Proposition 1 the constructor *never* returns more points than requested, so the realised population is a downward rounding of the requested one and the drift is one-signed. Second, it returns exactly the requested number for only 8.7% of (N, M) combinations, which we verified for $2 \leq M \leq 15$ and $M \leq N \leq 1000$; the round-down is therefore the common case rather than the exception.

4.2 How the affected files use the write-back

The lexical audit of Section 3.2 records what happens to the two outputs in each affected file. In 42 of the 84 affected files the returned set is used again after the assignment, most often as the reference set of the environmental selection, so there the population, the reference set and the per-generation cost all change together; in the remaining files

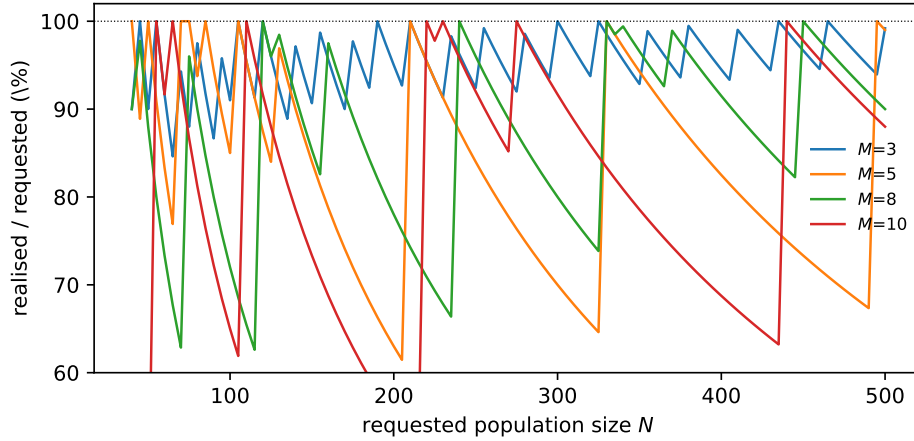


Figure 1 Realised population size as a fraction of the requested size for the default reference-vector constructor of PlatEMO 4.16. The realised count is a step function of the request; at the sizes used in practice it lies below the request, and the deficit grows with the number of objectives

Table 3 Realised population size returned by the default reference-vector constructor for requested sizes N and objective counts M . Entries equal to the request are set in bold. The constructor never exceeds the request and equals it only at admissible lattice sizes

$M \setminus N$	91	100	105	120	150	200	275	300	500
2	91	100	105	120	150	200	275	300	500
3	91	91	105	120	136	190	253	300	496
4	84	84	84	120	120	165	220	286	455
5	85	85	105	105	126	126	210	210	495
6	77	77	77	112	147	182	273	273	462
7	91	91	91	112	112	168	238	294	490
8	72	72	72	120	128	156	240	240	450
9	90	90	90	90	90	174	210	210	495
10	65	65	65	110	110	110	275	275	440

only the size field is overwritten, and the reference set is whatever the file already held. The distinction does not affect the drift, which is set by the assignment alone, but it does determine whether a repair may discard the returned set. The audit also orders the assignment against the population initialisation, because an assignment that runs after the population has already been created would change the size field without changing the first generation. The assignment precedes the initialisation in 62 of the 84 files (74%); it follows it in 6 (FDV, GLMO, WASF-GA, GWASF-GA, LCSA and POCEA), where the first generation is already at the requested size and only later generations are affected; and in 16 files the initialisation happens inside a helper routine that the audit does not resolve. We report the breakdown rather than a single number because the three groups behave differently, and the released per-file table gives the line numbers so that a reader can check each classification.

4.3 Realised count versus requested size

Table 3 and Figure 1 give the realised count for requested sizes up to 500 and objective counts from 2 to 10. The deficit is zero only when the request coincides with an admissible lattice size. At $N = 100$ and $M = 3$, the setting used in the great majority of three-objective studies, the algorithm runs with 91 individuals; at $M = 5$ with 85; at $M = 8$ with 72; at $M = 10$ with 65. The worst deficit in the table is 45%, and over the whole scanned range the worst case is larger still, reaching a deficit of 75% of the request — that is, realising as little as a quarter of what was asked — at high objective counts; the released table covers that range.

4.4 Census of the distribution

We searched the released tree. The platform ships 360 algorithms and 630 problems, organised as 360 algorithm directories, one per algorithm, holding 1887 .m source files; a single algorithm is split across a main file and its helper files, so the census reports both levels. Of the 1887 files, 84 (4.5%) contain the assignment, and they belong to 83 of the 360 algorithm directories, that is 23.1% of the algorithms in the release; 82 of the 83 contribute one file each and one algorithm contributes two, so $82 \times 1 + 1 \times 2 = 84$ files across 83 algorithms. The affected algorithms include the canonical NSGA-III and MOEA/D implementations and their descendants, RVEA and its variants, LMOCSO, HEA and C-TAEA, and the host itself, while 3 of the sixteen algorithms we traced (NSGA-II, SPEA2,

Table 4 Census of the PlatEMO 4.16 algorithm tree, at both the algorithm and the file level, for the assignment `[Z,Problem.N] = UniformPoint(Problem.N,Problem.M)`. The algorithm count is the number of directories containing at least one affected file

family	algorithms	files	examples
MOEA/D family	26	26	MOEA/D, MOEA/D-DRA, MOEA/D-DE, ...
NSGA-III family	4	4	NSGA-III, A-NSGA-III, DCNSGA-III, ...
RVEA family	4	4	RVEA, RVEAa, RVEA-iGNG, ...
Large-scale	14	15	LMOCSSO, HEA, C-TAEA, FDSEA
Other	35	35	AdaW, ParEGO, SPEA-R, ...
total	83	84	of 360 algorithms, 1887 files

Table 5 Realised cardinality of the reference set for the same nominal request in two libraries. The first row gives the requested size and the second the number of objectives; the last row is the ratio of the PlatEMO value to the pymoo value. The two agree when the request happens to be an admissible single-layer lattice size and diverge otherwise

library	100	100	100	100	200
(N_{req})	3	5	8	10	3
PlatEMO 4.16	91	85	72	65	190
pymoo 0.6.2	91	70	36	55	190
ratio	1.00	1.21	2.00	1.18	1.00

IBEA) are unaffected and run at the requested size, which is what makes the comparison in Table 2 a confounded one. Table 4 groups the affected algorithms by family. The assignment is not a rare idiom confined to one author, which is why the framework rather than the incident is the contribution.

4.5 The same request in another library

The framework claims to be portable, so we ran the audit on a second library rather than asserting that it would apply. pymoo is a widely used Python framework with a reference-direction module of its own, and it exposes the same two constructions: a Das–Dennis lattice and a factory that returns an exact number of directions. The audit’s trigger, a routine that returns a reference set together with a size that is then written into a shared population-size field, is *absent* there: the Das–Dennis entry point takes the partition count and derives the cardinality, and the exact-count factory is a separate call, so no user-set population size is overwritten. The audit therefore returns a negative result on pymoo [25], which is what makes it a test rather than a tautology.

The comparison in Table 5 is therefore not a second defect report. What is *not* absent is the underlying arithmetic, and the difference between the two libraries is instructive: the table compares the realised cardinality for the same nominal request. At three objectives both libraries return 91, because the lattice is the same; at five, eight and ten objectives they diverge, by factors of 1.21, 2.00 and 1.18. At eight objectives a study that declares $N = 100$ realises 72 directions in PlatEMO and 36 in pymoo, a factor of 2.00. The divergence is not a defect of either library: PlatEMO rounds down to the largest admissible lattice and then adds a second layer, while pymoo chooses the partition count whose cardinality is closest to the request and stops there. The point for the framework is that the nominal size does not determine the realised size even across two libraries that implement the same construction, let alone across algorithms within one library, and that a comparison table reporting only the nominal size is therefore unverifiable by construction.

5 Empirical consequences of the override

5.1 Experimental setup

We instantiate the framework on a host and a suite large enough for the effect to be measurable. The host is FDSEA [26], a recent large-scale reference-vector method that contains the assignment. Its published generation loop is governed by three constants: the number of individuals exchanged between its two subpopulations (n_{ex} , published default 10), the hybridisation weight that mixes a genetic and a differential operator (γ , published default 0.5 under an online update rule), and the neighbourhood size of the frequency-domain subpopulation (K , published default 5). We hold K at 5 and set the other two statically, so that the effect of the population size is separated from the effect of the constants. We compare nine configurations of the same host, all single-threaded, all with an evaluation budget of 10^5 and seeds $1, \dots, 30$ for the $D = 300$ problems and 20 for the $M = 5$ and $D = 1000$

problems: the host as published; the host with the size output discarded (repair 1); the host with the exactly- N constructor (repair 2); an adaptive scheduling layer that updates n_{ex} , γ and K online from a convergence and a diversity signal, used later only as a case study; and five static settings of n_{ex} and γ . The suite has 15 problems: DTLZ1, DTLZ2 and DTLZ7 [27] and the nine instances LSMOP1–LSMOP9 [28] at $M = 3$, $D = 300$; LSMOP1 and LSMOP6 at $M = 5$, $D = 500$; and LSMOP1 at $M = 3$, $D = 1000$. The indicator is the inverted generational distance [29] to a 400-point discretisation of the true front; for every front used here the front spans the unit interval in each objective, so the usual normalisation is the identity.

Two points about the design need to be stated plainly, because they determine how the results are to be read. First, the comparison isolates something narrower than the framework as a whole. All nine configurations are launched with the same requested size $N = 100$, and Section 5.2 adds a further arm, launched at the size it will realise, to separate the two effects below; the arm named “as published” realises $N_{\text{real}} = 91$ because the override takes effect, and the arm named “repair 1” realises $N_{\text{real}} = 100$. The measured difference is therefore the difference between the declared condition and the realised one, and by construction it coincides with the difference between a 91-individual and a 100-individual run of the host. We do not claim that fidelity has an effect separable from the population size; we claim, and measure, that declaring one size while running another changes the conclusion a study would draw. Second, the termination test is evaluated at generation boundaries, so a run that stops when the budget is exhausted can overshoot by less than one generation: the published arm consumes 100090 evaluations, 90 or 0.09% above the nominal budget, and the corrected arms consume 100000 exactly.

Algorithms share seeds, so the runs are paired and we use the two-sided Wilcoxon signed-rank test, reporting also the Holm–Bonferroni correction over the 15 comparisons, the sign test, and Cliff’s delta as an effect size that does not assume a distribution. Throughout, the central tendency is reported as the *median* over the runs, the interquartile range of the per-run record is used as spread, and every percentage change quoted in the text, in the tables and in Figure 2 is computed from medians as a ratio of medians, so that the three agree; optimisation runs occasionally produce a few dominating outliers, and the median is the robust summary. The raw per-run records are released, so the mean can be recovered exactly.

5.2 Three arms: declared, realised and repaired

Because the published configuration differs from the repaired one in the population actually evaluated, a comparison between those two alone cannot say whether the problem is the declaration or the size. We therefore ran a third arm in which the host is configured at the size it will realise — 91 at three objectives and 85 at five, which are the two values the constructor returns for a request of 100 at those objective counts (Table 3) — so that the declaration and the realisation agree. The three arms are then *published* (declared 100, realised 91), *declared* (declared 91, realised 91) and *repaired* (declared 100, realised 100). Table 6 gives all three pairwise comparisons, and each isolates a different thing: declared against published holds the realised size fixed at 91 and varies only the declaration, so it measures the *declaration* effect; repaired against declared varies the realised size from 91 to 100 at a fixed declaration, so it measures the *size* effect; and repaired against published is their combination, which is the quantity a reader of the published paper actually experiences.

Published and declared agree closely. The declared arm requests 91 at three objectives and 85 at five, which are the values the published arm realises; the two runs therefore evaluate the same number of individuals. Their *signed* median change is +0.0%, the median of the absolute changes is 1.2% and the largest is 4.2%, and the paired signed-rank test fails to separate them on 15 of the 15 problems: the smallest of the 15 p-values is 0.12, so no problem separates even before correction (Table S6 of the supporting information lists them). This is the direct evidence for the claim that motivates the paper. A run that declares 100 and a run that declares 91 are the same run, because both evaluate 91 individuals; whatever a study reports, the population it used is the one the constructor returned. The residual differences are of the order of one per cent and change sign across problems, which is what one expects when the only difference is that the two arms initialise different numbers of candidates before the first selection. Against that, repaired and declared differ by a median of 2.7%, which is the pure realised-size effect. Read against the published arm instead, the repaired host is better on 12 problems, identical on 1 and worse on 2, with a median of +1.7%; that is the count the abstract reports. The three arms therefore separate the two effects the single comparison conflates, and they locate the defect where the framework puts it: not in the algorithm, which behaves as the realised size dictates, but in the number printed beside the result.

5.3 Effect of restoring fidelity on the host

Table 6 reports the host on all 15 problems, Table S3 of the supporting information adds repair 2, and Figure 2 shows the per-problem median change. Restoring the requested size changes the median indicator from -4.1% to

Table 6 Three-arm comparison: published, declared and repaired, on the same host and seeds. Published declares 100 and realises 91; declared declares and realises 91 (the request is 91 at three objectives and 85 at five); repaired declares and realises 100. The three right-hand columns are median relative changes of the first-named arm against the second, positive meaning the first attains a smaller indicator, so that $decl - pub$ isolates the declaration effect at a fixed realised size, $rep - decl$ the realised-size effect at a fixed declaration, and $rep - pub$ their combination

problem	published	declared	repaired	decl-pub	rep-decl	rep-pub
DTLZ1	0.0657	0.0670	0.0584	-1.9	+12.8	+11.1
DTLZ2	0.0609	0.0620	0.0603	-1.8	+2.7	+1.0
DTLZ7	0.0590	0.0598	0.0543	-1.3	+9.1	+7.9
LSMOP1	0.1032	0.1035	0.0945	-0.3	+8.7	+8.4
LSMOP2	0.0812	0.0807	0.0823	+0.5	-2.0	-1.5
LSMOP3	0.8645	0.8645	0.8645	+0.0	+0.0	+0.0
LSMOP4	0.1448	0.1408	0.1424	+2.8	-1.2	+1.7
LSMOP5	0.3559	0.3554	0.3531	+0.1	+0.7	+0.8
LSMOP6	0.6429	0.6428	0.6363	+0.0	+1.0	+1.0
LSMOP7	0.8052	0.8052	0.8383	+0.0	-4.1	-4.1
LSMOP8	0.0769	0.0778	0.0748	-1.2	+3.9	+2.7
LSMOP9	0.7359	0.7358	0.7351	+0.0	+0.1	+0.1
LSMOP1 (5,500)	0.3971	0.3904	0.3736	+1.7	+4.3	+5.9
LSMOP6 (5,500)	0.9400	0.9001	0.8693	+4.2	+3.4	+7.5
LSMOP1 (3,1000)	0.0992	0.1013	0.0904	-2.2	+10.8	+8.9

11.1%, the median change over the suite being 1.7%. The direction of the effect is not uniform, and we state the counts exactly rather than in summary. These counts are against the published arm — the $rep - pub$ column of Table 6 — which measures the combined declaration-and-size effect; the pure realised-size effect ($rep - decl$) is reported in Section 5.2. The corrected host attains a smaller median indicator on 13 of the 15 problems and a larger one on the remaining 2. We give the breakdown rather than a single count, because one of the 13 is not a real improvement: twelve problems improve clearly, LSMOP3 is a tie in all but name with the two medians differing by about 3×10^{-6} so that the printed value is +0.0%, and two problems degrade (LSMOP2 and LSMOP7), both by a small margin; the pure size effect ($rep - decl$ in Table 6) additionally degrades LSMOP4, which the declaration effect partially offsets. Read as *no worse*, the count is thirteen; read as *improved*, it is twelve, and we use the latter in the abstract.

Statistical significance is weaker than the direction count, and we report it as such. The change is significant on 4 of the 15 problems at the uncorrected 0.05 level, and on 1 problem after Holm-Bonferroni correction; the single survivor is an $M = 5$ instance, where the realised deficit is largest. Power over fifteen comparisons is limited at thirty seeds, so we complement the tests with an effect size that does not depend on a distributional assumption: Cliff’s delta ranges in magnitude up to 0.64. Independently, a bootstrap confidence interval for the median difference excludes zero on 8 of the 15 problems. That count is larger than the corrected test because the interval is not adjusted for multiplicity, so we report it as exploratory and lead with the corrected count rather than with the interval. The honest reading is that the direction of the effect is consistent across the suite, that its magnitude is confidently non-zero on about half the problems, and that Proposition 1 bounds the sign rather than the size.

Two points deserve emphasis. First, the effect is an improvement and not a neutral re-ranking: the override under-estimates what the host achieves at its reported size, so a study that prints $N = 100$ and runs 91 individuals publishes a number that belongs to a smaller configuration. Second, the direction of the mean can disagree with the median and with the majority of runs when a few outliers dominate: on one problem the corrected host is better on 20 of 30 runs and by median, yet its mean is worse. This is precisely why a single mean hides the effect.

5.4 Two ways of restoring fidelity

Table S3 of the supporting information gives repair 2 alongside repair 1. The two repairs are not equivalent: repair 2 changes the reference set as well as the population, and on several problems the reference set produced by the default constructor is at least as good even though it is smaller. Where the algorithm permits the population to differ from the reference set, repair 1 is the conservative choice, because it changes exactly one thing; which of the two is admissible depends on the algorithm, as Section 5.6 shows.

5.5 A reported gain undone by the override

To show how the override manufactures a gain we use the adaptive scheduling layer that originally motivated this work; it drives n_{ex} , γ and K online from a convergence and a diversity signal. It is a case study, not a contribution, and we report it on all 15 problems rather than on a favourable subset (Table S5 of the supporting information).

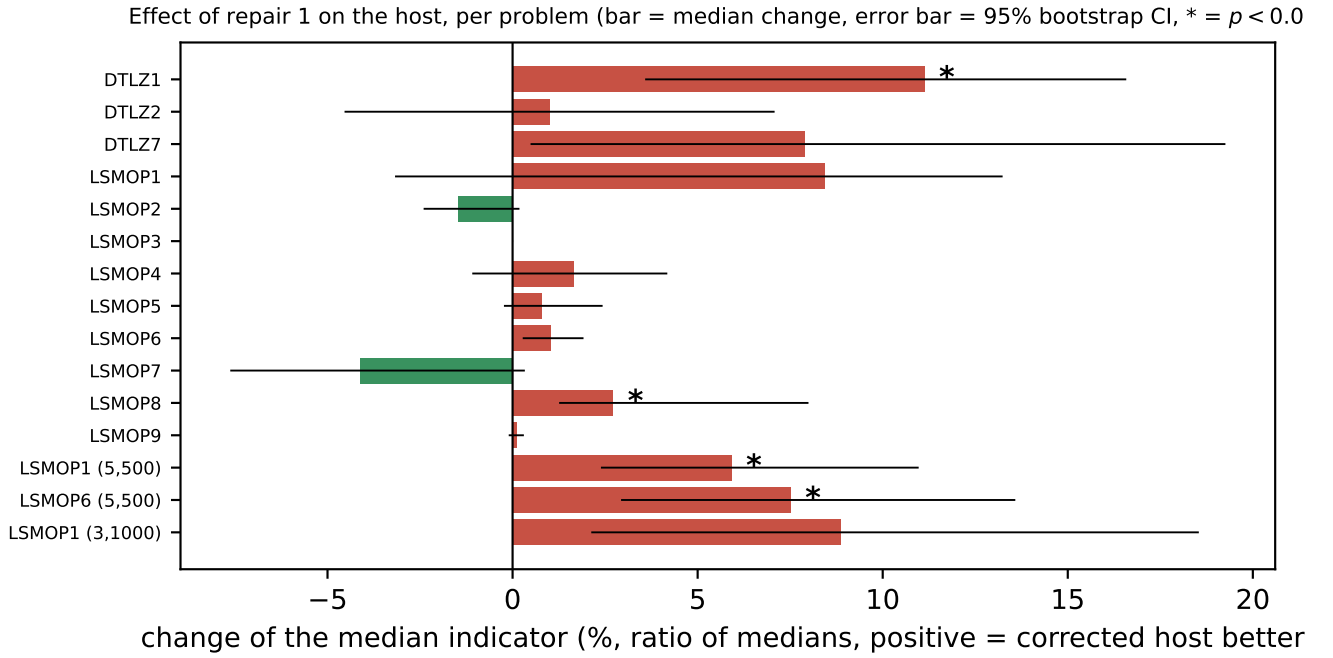


Figure 2 Change of the median indicator of the host from restoring the requested population size, shown separately for each of the fifteen problems; the bar is the ratio of medians, the same quantity as the repaired column of Table 6, the error bar is a bootstrap 95% confidence interval for the median difference, and an asterisk marks $p < 0.05$ before correction

Table 7 Wilcoxon signed-rank test of repair 1 against the published host. The baseline is the published run, which Section 5.2 shows is statistically the same as the declared arm (0/15 problems separate; Table S6), so this table measures the combined declaration-and-size effect; Section 5.2 separates the two. “Repair 1 wins” counts the runs on which the corrected host attains a smaller indicator

problem	repair 1 wins	p (signed rank)	Holm–Bonferroni	sign test	Cliff δ
DTLZ1	21/30	0.0155	0.1855	0.0428	+0.45
DTLZ2	18/30	0.3387	1.0000	0.3616	+0.14
DTLZ7	20/30	0.0879	0.7914	0.0987	+0.32
LSMOP1	18/30	0.0549	0.6041	0.3616	+0.33
LSMOP2	12/30	0.1840	1.0000	0.3616	-0.20
LSMOP3	14/30	0.7151	1.0000	0.8555	-0.02
LSMOP4	15/30	0.4522	1.0000	1.0000	+0.15
LSMOP5	19/30	0.4400	1.0000	0.2005	+0.23
LSMOP6	19/30	0.0699	0.6989	0.2005	+0.37
LSMOP7	15/30	0.3931	1.0000	1.0000	-0.23
LSMOP8	22/30	0.0099	0.1291	0.0161	+0.45
LSMOP9	17/30	0.8078	1.0000	0.5847	+0.13
LSMOP1 (5,500)	19/20	$< 10^{-4}$	$2.9e-04$	$< 10^{-4}$	+0.59
LSMOP6 (5,500)	17/20	0.0037	0.0512	0.0026	+0.64
LSMOP1 (3,1000)	14/20	0.0973	0.7914	0.1153	+0.44

Against the published host the layer is better on 10 problems and worse on the other 5, with a median gain of only +0.87%: the double-digit gains that motivated it are concentrated in 6 problems and absent elsewhere. Against the host at the requested population size, which holds the realised size fixed at 100 and therefore isolates the scheduling, the picture narrows further: the layer is better on 8 problems and worse on 7, median +0.04%. On the 6 problems carrying a gain above 5%, matching the size removes 0.9 to 8.8 percentage points of it, so much of the apparent advantage is the population difference rather than the scheduling; elsewhere no gain exceeds 8.6% in either direction. The layer and the static settings of Section 5.7 both run at the requested population size and are directly comparable, and against the best static setting the layer attains a smaller median on only 2 of the 15 problems, including 2 of the 6 where its gain over the published host exceeds 5%. We therefore do not claim that the layer is worthless; we claim that the evidence which previously supported it was largely an artefact of comparing a 100-individual layer against a 91-individual host, and that at a matched size the advantage is small and inconsistent in sign.

Table 8 NSGA-III with the published override and with the requested population size restored by repair 1, on six large-scale problems. NSGA-III is case A, so repair 1 leaves the reference set untouched; the realised population is 91 in the published row and 100 after the repair, as in Table 6

NSGA-III (median)	LSMOP1	LSMOP2	LSMOP4	LSMOP6	LSMOP8	LSMOP9
as published	0.6288	0.0829	0.1963	2.9299	0.3459	1.0834
repair 1 (requested N)	0.5832	0.0836	0.1999	3.5033	0.3437	1.0770
p (signed rank)	0.0841	0.0155	$6.7e - 04$	0.5291	0.9354	0.0767

5.6 Consequence across algorithms

The affected set is not confined to one family. A comparison table that mixes affected and unaffected methods is therefore confounded, because the affected methods run with fewer individuals than they report. Table 8 quantifies this on NSGA-III. The picture is more equivocal than for the host: the corrected NSGA-III is better on LSMOP1 but worse on LSMOP4 and LSMOP6, and the signed-rank test is significant on LSMOP2 and LSMOP4 even where the medians barely move, because the override also changes the shape of the run-to-run distribution rather than only its centre. There is accordingly no uniform sign to the bias.

We attempted the same repair on the other affected peers, and this is where the catalogue of Section 3.3 becomes concrete. NSGA-III is case A and accepts repair 1. MOEA/D is case B: it indexes its individuals by reference vector, and repair 1 fails with an index-out-of-bounds error, while repair 2 restores it. RVEA and LMOCSO [30] are case C: both compute, inside their environmental selection, the smallest inter-vector angle $\gamma_i = \min_j \arccos(1 - d_{\cos}(v_i, v_j))$, where $d_{\cos}$ is the cosine distance, so the argument of $\arccos$ is exactly the cosine, and the convergence term is divided by γ_i . The mechanism is the one measured in Table 1: the exactly- N constructor reduces the smallest angle of the reference set, by a factor of 1.45 at $M = 3$ and 2.31 at $M = 5$, so γ_i falls and the convergence term is amplified in proportion. This is a quantitative degradation of the selection criterion rather than the degeneracy that an earlier formulation of this case assumed, and it is why the case needs its own repair rather than either of the two previous ones. Repair 1 fails still earlier for RVEA, because RVEA's reference-vector adaptation writes $V(1 : N, :)$ and therefore requires the vector set to have at least N rows. Fidelity can accordingly be restored everywhere, but not uniformly, and the admissible repair has to be selected from the coupling between population and reference set rather than applied as a universal rule.

5.7 Static settings and cost

Sweeping the two constants statically (Table S1 of the supporting information) suggests that the exchange count is the load-bearing one, and it provides the reference against which the layer of Section 5.5 is judged; we report it only as context for the case study, and it is therefore placed in the supporting information rather than here. Table 9 reports the population, the evaluations consumed, the generations completed and the wall-clock time. The host as published completes 552 generations and the corrected host 499, a ratio of 1.106 against the ratio of realised population sizes, 1.099, as Proposition 2 predicts. The small excess over the predicted ratio comes from the evaluation accounting rather than from the population: the host keeps two subpopulations and consumes about $2N$ evaluations per generation, so one generation of the published arm costs fewer evaluations than one generation of the corrected arm, and the termination test, which is evaluated at generation boundaries, lets the published run stop 90 evaluations (0.09%) past the nominal budget. The wall-clock differences are a consequence of the same effect and are not evidence about the algorithm.

The same static family lets us check that the comparison between the layer and the static settings of Section 5.5 is made over the same search space, since the layer updates K online while the static settings used so far held it at its published value. Table S2 of the supporting information closes that gap by sweeping $K \in \{3, 5, 10\}$ at a fixed exchange count and hybridisation weight. The sensitivity is not marginal. The best value differs across problems (5 problems prefer $K=3$, 3 prefer $K=5$ and 7 prefer $K=10$), with a median spread of 9.6% of the indicator between the best and the worst of the three settings. A median spread understates the case, however, so we report the exception rather than let the summary hide it: on DTLZ1, $K=10$ yields a median indicator of 1116.0 against 0.0418 for $K=3$, a factor of 26723, and the failure is not one unlucky seed, since 29 of the 30 runs at that setting exceed unity. The evaluation counts are correct (Table 9 uses the same budget), so the runs completed; they simply do not

Table 9 Population, evaluations consumed, generations completed and wall-clock time

configuration	population	evaluations	generations	wall (s)
as published	91	100090	552	18.9
requested N	100	100000	499	18.5

converge, and on 1 of the 15 problems the choice of K changes the outcome by more than an order of magnitude. This matters for the case study that motivates the paper, because the adaptive layer updates K online over a range that contains the failing value, so a static choice of K cannot be treated as neutral in either direction.

6 Discussion and conclusion

We have argued that the population size of a reference-vector MOEA is a controlled quantity that can silently stop being controlled, and we have given a framework that makes the failure detectable, repairable and reportable. Applied to PlatEMO 4.16, the framework reveals that 84 of 1887 algorithm source files, belonging to 83 of 360 algorithms, overwrite the requested population size with the size of an approximate reference lattice that never exceeds the request and is exact in only 8.7% of cases; at the sizes in common use the reduction reaches 45%. On a large-scale suite, restoring the requested size changes the host’s median indicator on 13 of 15 problems — with twelve clear improvements, one tie and two degradations — and an exploratory, uncorrected bootstrap confidence interval excluding zero on 8; it accounts for most of the median scheduling gain that had been attributed to the scheduler, leaving a small and sign-inconsistent remainder; and it changes sign across peers, so that no single repair is universally admissible.

Two conclusions follow, and they are methodological rather than algorithmic. The first is that the framework converts a diffuse worry about experimental fairness into a procedure with four checkable outputs: an audit that yields a census at the algorithm and file level, a deficit table, a repair catalogue with the mechanism of the hardest case measured, and a reporting protocol. Any library that returns a reference set together with a size can be audited in the same way, and any of the three repair cases can be recognised from the coupling between population and reference set rather than by trial and error. The second is that the finding should be read as a reporting concern and not as a defect in the algorithms: the rounding performed by a simplex lattice is unavoidable, the write-back is often deliberate, and no run in this study is in any way broken. What is avoidable is publishing a table that states a size the run did not use, which is the one thing the framework asks a study not to do. The recent finding that thirteen widely used frameworks differ in ways that change comparison outcomes, even at identical settings, indicates that the audit is worth running more broadly [10].

The framework is deliberately stated so that it does not depend on PlatEMO. The trigger it looks for is a constructor that returns a reference set together with a size which is then written into a shared population-size field, and this pattern is not specific to one library: it appears wherever a convenience routine both samples reference vectors and reports how many it sampled. The audit of Algorithm 1 can therefore be applied unchanged to any such library, and the deficit function δ can be tabulated once per constructor. The three repair cases are likewise recognised from the coupling between population and reference set rather than from the identity of the algorithm, so a practitioner who can answer the single question “must the population equal the reference set?” can select the admissible repair.

The study has clear limits. The audit covers one distribution at one version, so the census measures PlatEMO 4.16 and is not a survey; extending it is a necessary next step before the portability of the pattern can be asserted rather than argued. The empirical magnitude is host- and suite-dependent, and the comparison between the declared and the realised configuration coincides by construction with a difference in population size, so we quantify a reporting error rather than an intrinsic property of fidelity. The $M = 5$ and $D = 1000$ parts of the suite are smaller than the $D = 300$ part, and thirty seeds give limited power over fifteen simultaneous comparisons, which is why we report effect sizes alongside tests. The case-study layer is one design; we do not claim that no scheduler of this family could add value, only that the evidence for this one was confounded and that at a matched size it is small. Finally, the framework detects and repairs infidelity but cannot decide how large a population should be, which remains a modelling choice for the practitioner.

Data availability. The release is publicly available at github.com/qingdi-miruisi/population-size-fidelity, at tag v1.2-final, which resolves to a fixed commit, and contains the audit script of Algorithm 1, the per-file audit listing with line numbers, the complete realised-count table $L(N, M)$ over 13895 combinations, the three-quantity trace of Table 2, the cross-library comparison of Table 5, the corrected variants for the three repair cases, the driver scripts that regenerate every table and figure of this paper, and the raw per-run archives from which all reported statistics

are recomputed. No file of the audited distribution was modified. All experiments were run in MATLAB R2026a against PlatEMO 4.16 with the seeds recorded in the source, are single-threaded. Every table and figure can be regenerated from the released scripts and raw per-run archives.

Conflict of interest. The author declares that he has no conflict of interest. The audited platform is an open-source community project; the author has no financial or personal relationship with its developers, and no file of the audited distribution was altered in the course of this study.

Author contributions. Y. X. Zhang is the sole author and is responsible for the conceptualisation, the formal analysis, the software, the experiments, the validation and the writing of the manuscript.

Acknowledgements This research received no external funding. The author thanks the developers of PlatEMO for making the platform and its source code openly available, which is what makes an audit of this kind possible.

Supporting information Table S1, the five static settings of the exchange count and the hybridisation weight; Table S2, the sensitivity of the host to the neighbourhood size K ; Table S3, the host with repair 1 and repair 2 side by side; Table S4, the bounded-angle constructor of Case C on four settings; Table S5, the adaptive scheduling layer against both references on all fifteen problems; the p-values of Table S6; the parser test suite; the per-file audit listing with line numbers; the complete realised-count table $L(N, M)$ as a CSV file; the measured inter-vector angles; the corrected variants for the three repair cases and the raw per-run records. The supporting information is available online at info.scichina.com and link.springer.com. The supporting materials are published as submitted, without typesetting or editing. The responsibility for scientific accuracy and content remains entirely with the authors.

References

- 1 Deb K, Jain H. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. *IEEE Trans Evol Comput*, 2014, 18: 577–601
- 2 Zhang Q, Li H. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Trans Evol Comput*, 2007, 11: 712–731
- 3 Cheng R, Jin Y, Olhofer M, et al. A reference vector guided evolutionary algorithm for many-objective optimization. *IEEE Trans Evol Comput*, 2016, 20: 773–791
- 4 Wu X, Wu S H, Wu J, et al. Evolutionary computation in the era of large language model: survey and roadmap. *IEEE Trans Evol Comput*, 2024, 29: 177–208
- 5 Liu F, Lin X, Yao S, et al. Large language model for multiobjective evolutionary optimization. In: *Proceedings of the International Conference on Evolutionary Multi-Criterion Optimization*, Singapore, 2025. 178–191
- 6 Huang Y, Wu S, Zhang W, et al. Autonomous multi-objective optimization using large language model. *IEEE Trans Evol Comput*, 2025, in press
- 7 Liu J, Sarker R, Elsayed S, et al. Large-scale evolutionary optimization: a review and comparative study. *Swarm Evol Comput*, 2024, 85: 101466
- 8 Wang P, Wu X, Deng H. Large-scale multi-objective optimization algorithms: a decade survey. *Expert Syst*, 2025, 42: e70157
- 9 Ishibuchi H, Pang L M, Shang K. Population size specification for fair comparison of multi-objective evolutionary algorithms. In: *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, Toronto, 2020. 1095–1102
- 10 Ravber M, Smid M, Moravec M, et al. Performance comparison of single-objective evolutionary algorithms implemented in different frameworks. *Informatica*, 2025, 36: 677–712
- 11 Tian Y, Cheng R, Zhang X, et al. PlatEMO: a MATLAB platform for evolutionary multi-objective optimization. *IEEE Comput Intell Mag*, 2017, 12: 73–87
- 12 Lopez-Ibanez M, Branke J, Paquete L. Reproducibility in evolutionary computation. *ACM Trans Evol Learn Optim*, 2021, 1: 14:1–14:21
- 13 Yang L, Zhang Y, Cao J, et al. A many-objective evolutionary algorithm based on reference vector guided selection and two diversity and convergence enhancement strategies. *Appl Soft Comput*, 2024, 154: 111369
- 14 Derrac J, Garcia S, Molina D, et al. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol Comput*, 2011, 1: 3–18
- 15 Demsar J. Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res*, 2006, 7: 1–30
- 16 Lian J J. Time-fair benchmarking for metaheuristics: a restart-fair protocol for fixed-time comparisons. *arXiv:2509.08986*, 2025
- 17 Molina D, Del Ser J, Poyatos J, et al. The paradox of success in evolutionary and bioinspired optimization: revisiting critical issues, key studies, and methodological pathways. *Swarm Evol Comput*, 2025, 98: 102063
- 18 Kononova A V, van Stein N, Mersmann O, et al. Benchmarking that matters: rethinking benchmarking for practical impact. *arXiv:2511.12264*, 2025
- 19 Yao S, Liu F, Lin X, et al. Multi-objective evolution of heuristic using large language model. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, Philadelphia, 2025. 27144–27152
- 20 Zhou M C, Cui M, Xu D, et al. Evolutionary optimization methods for high-dimensional expensive problems: a survey. *IEEE/CAA J Autom Sinica*, 2024, 11: 1092–1105
- 21 Liang J, Lou Y, Yu M, et al. A survey of surrogate-assisted evolutionary algorithms for expensive optimization. *J Membr Comput*, 2025, 7: 108–127
- 22 Liu Z, Han F, Ling Q, et al. Constraint-Pareto dominance and diversity enhancement strategy-based evolutionary algorithm for solving constrained multiobjective optimization problems. *IEEE Trans Evol Comput*, 2025, 29: 2771–2784
- 23 Tian Y, Xiang X, Zhang X, et al. Sampling reference points on the Pareto fronts of benchmark multi-objective optimization problems. In: *Proceedings of the IEEE Congress on Evolutionary Computation*, Rio de Janeiro, 2018. 1–6
- 24 Das I, Dennis J E. Normal-boundary intersection: a new method for generating the Pareto surface in nonlinear multicriteria optimization problems. *SIAM J Optim*, 1998, 8: 631–657
- 25 Blank J, Deb K. Pymoo: multi-objective optimization in Python. *IEEE Access*, 2020, 8: 89497–89509
- 26 Wang W, Tan Z, Wang Y, et al. Enhancing the scalability of large-scale multi-objective evolutionary algorithm through frequency domain search. *Swarm Evol Comput*, 2026, 106: 102423
- 27 Deb K, Thiele L, Laumanns M, et al. Scalable multi-objective optimization test problems. In: *Proceedings of the IEEE Congress on Evolutionary Computation*, Honolulu, 2002. 825–830
- 28 Cheng R, Jin Y, Olhofer M. Test problems for large-scale multiobjective and many-objective optimization. *IEEE Trans Cybern*, 2017, 47: 4108–4121
- 29 Zitzler E, Thiele L. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE Trans Evol Comput*, 1999, 3: 257–271
- 30 Tian Y, Zheng X, Zhang X, et al. Efficient large-scale multi-objective optimization based on a competitive swarm optimizer. *IEEE Trans Cybern*, 2020, 50: 3696–3708